

Apostila Vue.js — Aula 3: Componentização

Oct 2, 2026 · @Someone

Resumo do conteúdo

Componentes dividem a interface em peças menores e reutilizáveis: dados descem do pai para o filho por **props** e avisos sobem do filho para o pai por **emits**.

O que é um componente

Um componente é uma mini aplicação Vue, com seu próprio `template`, `data()` e `methods`. Ele é registrado com `app.component()` **antes** de `app.mount()` e usado como uma tag HTML.

```
app.component("tarefa-item", {
  props: ["texto"],
  template: `<li>{{ texto }}</li>`
});
```

```
<tarefa-item texto="Estudar Vue"></tarefa-item>
```

props: do pai para o filho

Props funcionam como parâmetros de uma função. Sem dois-pontos, o valor chega como texto fixo (`texto="Estudar Vue"`); com dois-pontos, é uma expressão JavaScript (`:feita="true"`, `:preco="10"`, `:texto="t.texto"`).

A forma recomendada de declarar props inclui validação:

```
props: {
  texto: { type: String, required: true },
  feita: { type: Boolean, default: false }
}
```

Regra de ouro: o filho nunca altera uma prop (nada de `this.texto = ...`). O dono do dado é sempre o pai.

emits: do filho para o pai

O filho avisa o pai com `$emit("nome-do-evento", valorOpcional)`. O pai escuta com `@nome-do-evento` e decide o que fazer.

```
app.component("botao-somar", {
  emits: ["clicou"],
  template: `<button @click="$emit('clicou')">+1</button>`
});
```

```
<botao-somar @clicou="incrementar"></botao-somar>
```

Fluxo de dados

Dados descem por props, eventos sobem por emits. Nunca o contrário: o filho jamais escreve direto em um dado do pai.

Exercícios

Resolva os exercícios abaixo usando o Vue via CDN, com um `index.html` e um `app.js` para cada um. Mantenha o console do navegador aberto (F12) para acompanhar os avisos do Vue.

Exercício 1 — Meu primeiro componente

Crie um componente `saudacao-usuario` que exiba um título `<h2>` com o texto "Olá, bem-vindo ao curso!". Use esse componente três vezes dentro de `#app`.

Exercício 2 — Saudação personalizada

Altere o componente do exercício 1 para que ele receba a prop `nome` e exiba "Olá, {nome}!". Use o componente três vezes, cada uma com um nome diferente.

Exercício 3 — Cartão de produto

Crie o componente `cartao-produto`, que recebe as props `nome`, `preco` e `emEstoque`. O cartão deve exibir o nome do produto, o preço no formato "R\$ 0,00" e o texto "Disponível" ou "Esgotado". Mostre na tela pelo menos três produtos diferentes.

Exercício 4 — Validando props

Reescreva as props do exercício 3 na forma com validação, seguindo as regras:

- `nome`: texto, obrigatório.
- `preco`: número, com valor padrão 0.
- `emEstoque`: booleano, com valor padrão `true`.

Depois, use o componente sem informar o `nome` e, em outro uso, passe um preço inválido. Anote os avisos que aparecem no console e explique o que cada um significa.

Exercício 5 — Lista de tarefas

No componente pai, crie uma lista com pelo menos quatro tarefas, cada uma com `id`, `texto` e `feita`. Exiba cada tarefa com um componente `tarefa-item`. As tarefas concluídas devem aparecer riscadas.

Exercício 6 — Concluindo tarefas

Continue o exercício 5. Ao clicar em uma tarefa, ela deve alternar entre feita e pendente. O componente filho não pode alterar suas props: quem muda o dado é o pai.

Exercício 7 — Botão reutilizável

Crie um componente `botao-acao` que recebe um rótulo e avisa o pai quando é clicado. No pai, exiba uma pontuação e use o mesmo componente três vezes:

- "+1" soma 1 ponto.
- "+10" soma 10 pontos.
- "Zerar" volta a pontuação para 0.

Exercício 8 — Contador com limite

Crie o componente `contador-controle`, com um botão "-", o valor atual e um botão "+". O valor fica guardado no pai, que não deve permitir números menores que 0 nem maiores que 10. Quando um limite for atingido, o botão correspondente deve ficar desativado.

Exercício 9 — Adicionando tarefas

Crie o componente `nova-tarefa`, com um campo de texto e um botão "Adicionar". Ao clicar, o texto digitado deve ser enviado ao pai, que inclui uma nova tarefa na lista do exercício 5. O campo deve ser limpo após adicionar, e textos vazios devem ser ignorados.

Exercício 10 — Desafio: mini app de tarefas

Monte uma aplicação completa de tarefas com três componentes:

Componente	O que faz
<code>nova-tarefa</code>	Permite criar uma nova tarefa

Componente	O que faz
<code>task-item</code>	Exibe uma tarefa, com opções para concluir e remover
<code>task-summary</code>	Mostra "X de Y tarefas concluídas"

Requisitos:

- ☐ Apenas o componente pai guarda a lista de tarefas.
- ☐ Todas as props são declaradas na forma com validação.
- ☐ Todo componente que dispara eventos os declara em `emits`.
- ☐ O número de tarefas concluídas é calculado com `computed`.
- ☐ Nenhum filho altera uma prop, e o console fica sem avisos.

Ao final, desenhe a árvore de componentes da sua aplicação, indicando quais dados descem por props e quais eventos sobem por emits.